

ECHO ORB

Game Design & Production Brief



BSc (Hons) in Applied Computing

Sinead Cleary (20102299)

Contents

1. Game Overview	3
2. Concept Summary.....	4
3. Unique Selling Points	5
4. Core Gameplay Loop.....	6
5. Key Features.....	7
5.1 Core Features.....	7
5.1.1 Main Gameplay.....	7
5.1.2 Editor	7
5.2 Secondary Features.....	7
5.2.1 Main Gameplay.....	7
5.2.2 Editor	8
5.3 Stretch Features	8
5.3.1 Main Gameplay.....	8
5.3.2 Editor	8
6. Look and Feel.....	9
6.1 Visual Style	9
6.2 Audio Style and Mood	10
7. Narrative, theme, and aesthetics	11
7.1 Story	11
7.2 Setting and Characters.....	11
7.3 Tone and themes.....	11
7.4 Aesthetics.....	11
8. Technical Approach.....	12
8.1 Game Engine:.....	12
8.2 Programming Language:	12
8.3 Key Systems:.....	12
8.3.1 Core Architecture.....	12
8.3.2 Level Editor Implementation.....	12
8.3.3 Puzzle Logic and Echo System	12
8.3.4 Game AI	12
8.3.5 Performance and Optimisation	12
8.3.6 UI and Accessibility.....	12
8.4 Tools:	12
8.5 Data Storage:	13
8.6 Version Control:	13

8.7 Deployment Platform:.....	13
9. Literature and Research Summary.....	14
9.1 Emergent Gameplay	14
Emergent Gameplay in Echo Orb	14
9.2 Human-Computer Interaction	15
HCI in Echo Orb	15
9.3 User Generated Content	16
UGC in Echo Orb	16
10. Scope and Feasibility.....	17
11. Production Plan	18
11.1 Methodology.....	18
11.2 Tools.....	18
11.3 Sprint Schedule	18
11.4 Gantt Chart.....	19
11.5 Risk Analysis.....	19
11.6 Testing Plan	19
12. Vertical Slice Specification.....	20
12.1 Mechanics and systems	20
12.2 Levels	20
12.3 Temporary Assets.....	20
12.4 Known Exclusions	20
References	22
Credits	23

1. Game Overview

- **Game Title:** Echo Orb
- **Tagline:** Uncover the echoes of adventure
- **Name:** Sinéad Cleary
- **Course:** Applied Computing
- **Supervisor:** Brendan Lyng
- **Target Platform:** PC

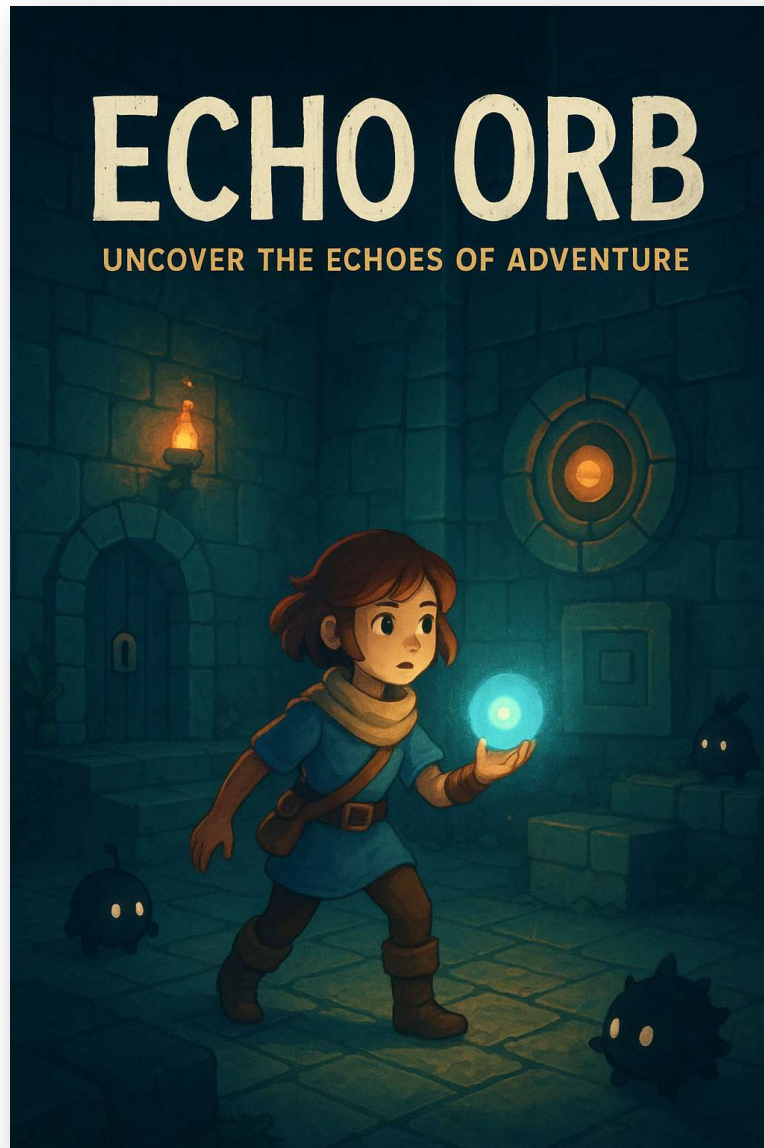


Figure 1. Echo Orb concept art generated with ChatGPT

2. Concept Summary

Echo Orb is a puzzle-action-adventure game for PC. The game is about main character Eli's quest through an underground hidden dungeon. They quickly learn that this is a place like no other, where an undiscovered civilization of unusual creatures roams the winding corridors and ominous chambers. Eli discovers that this place is controlled by a magical item: the echo orb. Eli uses the powerful force of the echo orbs to not only navigate, but also control the seemingly enchanted dungeon, and the interconnected systems within.

The game combines puzzle solving with action game elements. The puzzle aspects of the game include figuring out the correct combination and sequence of moves that will lead to making progressing to the next level or the next section of the level. As with any puzzle, players may need to experiment with many different combinations to find one that works. The action aspects include fighting hostile enemies, making quick and precise movements, and dodging attacks. This blend of fast-paced action with logical thinking creates a well-balanced mix of high intensity and slower, more thoughtful gameplay moments. The slower-paced parts of the game allow for a sense of relief, while the fast-paced ones bring thrill and excitement.

The central Echo Orb/Holder mechanic is what makes this game unique. It works both with and against the player, serving as both a puzzle solving tool, and a part of the puzzle itself. The player's aim is to traverse through each level from beginning to exit. They must do so by collecting Echo Orb items; powering and activating Echo Holders, which control mechanisms such as ghost walls, cannons, teleportation, and enemies; unlocking new rooms; and defending themselves from danger. As they player progresses, they will come across many different types of NPCs, some of which are hostile, some of which are friendly, and some of which can be made hostile due to the player's actions. Each of the different NPC types have unique characteristics, and each display different kinds of behaviour.

Aside from the main gameplay, the game will include a level editor, allowing players to use the existing game elements to create their own levels. This gives more creative players the ability to make something of their own directly within the game. Due to the modular nature of the puzzle elements, players may find new ways of combining the games mechanics to create new and unique scenarios that are not found in the base game levels.

Echo Orb is aimed at teenagers and adults who enjoy creative problem solving and engaging puzzles, as this game makes you think. It's also aimed at creative people who like to build their own levels. Similar games include *Baba Is You* for its puzzle elements, *The Binding of Isaac's* dungeons and combat, the *Super Mario Maker* series for its focus on creating your own levels, and *Geometry Dash* for its level editor. Fans of these games can all find something to enjoy in *Echo Orb*.

PC is the primary platform for this game, with keyboard and mouse being the ideal controls, especially for the level editing elements. Controller support is a potential future consideration.

The player should feel excited and engaged when engaging in high energy action moments, curious when exploring the vast dungeons, clever as they solve intricate puzzles and experiment with Echo Orb mechanics, and creative and expressive when building and designing their own levels. The mix of elements brings a wide range of emotions and feelings to the player experience.

3. Unique Selling Points

The following is a list of what sets Echo Orb apart from similar products:

- **Echo orb/holder system:** a unique puzzle mechanic where one action triggers all activated “holders”, each with their own actions.
- **Ghost wall mechanic:** walls are turned “on” or “off” with the echo system
- **Unique combat system:** combat is carried out through the echo system
- **Puzzle-action-adventure combination:** the game combines puzzle, action, and adventure elements
- **Interesting enemy AI:** multiple different types of enemies, each with their own separate behaviours and quirks
- **Create your own levels:** create your own levels with the in-game level editor
- **Cartoony art style:** original art created by me

4. Core Gameplay Loop

The core gameplay loop is:

collect orb → power holder → activate all powered holders → progress.

The player will have to repeat this sequence of actions multiple times to complete the level. Orbs are needed to power holders, holders can only be activated when powered, and progress is made by activating the correct combination of holders.

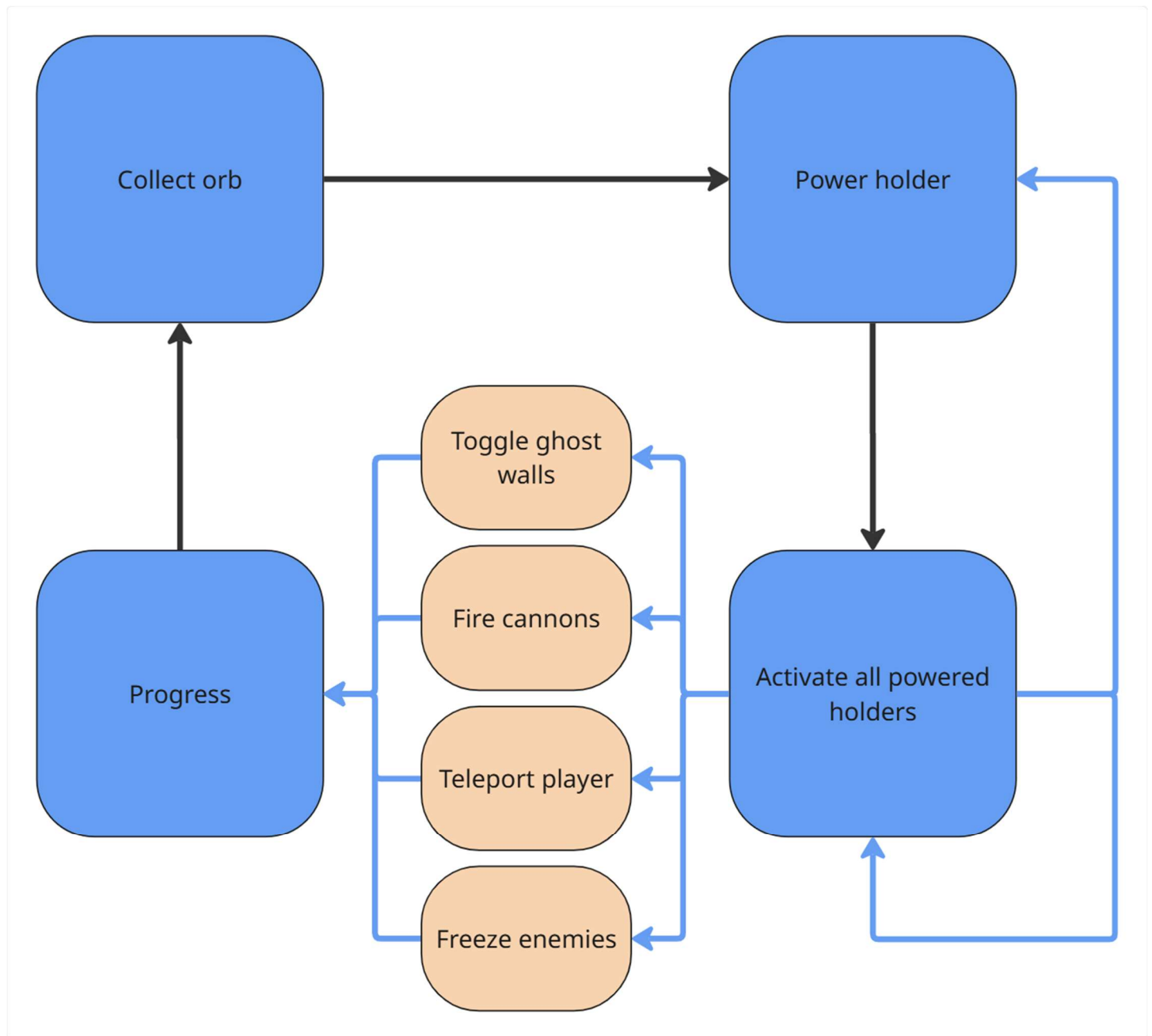


Figure 2. Core gameplay loop diagram created with Miro

5. Key Features

The game's key features are broken up into core, secondary, and stretch features. Core features are the most important, required features that will be developed first. Secondary features are the features that expand on the core features adding more interest, complexity, and variation. Stretch features are those that would be nice to have, but if time is limited, they may be cut from the final game. Each set of features is divided between main gameplay and editor features.

5.1 Core Features

5.1.1 Main Gameplay

- Player movement: move in all directions and collide with other objects
- Pickup orb: pickup orbs and add to inventory
- Place orb in holder: place orb in a holder (if there is an orb in the inventory) and remove from inventory
- Ghost walls: walls can be activated/deactivated with Echo system
- Cannons: cannons can shoot enemies and player
- Echo orb/holder activation system: if a holder is powered it can be activated from anywhere in the level
- Echo holders:
 - Ghost wall holders (multiple colours): differently coloured holders activate/deactivate differently coloured walls
 - Cannon holder: shoot cannons
- Stairs transition between levels
- Level restart system: needed to prevent the player from becoming soft locked (stuck in a certain part of the level in a way that it is impossible to progress).
- NPCs:
 - Simple enemy follows and attacks player
- Enemies drop orbs: Some enemy types drop orbs when killed
- ~5 premade levels plus cutscenes
- UI: Heads Up Display shows current player stats (health, etc.) and currently powered orbs so the player can keep track. Menus including main menu, settings, and pause.

5.1.2 Editor

- Place tiles
- Place objects (orbs, holders, walls, NPCs, cannons, etc.)
- Remove objects
- UI for the above editor features

5.2 Secondary Features

5.2.1 Main Gameplay

- More NPC types:
 - Friendly creature talks to the player revealing background story and gameplay tips
 - Disguised enemy: an enemy that disguises itself as an orb, tricking the player and attacking when the player gets close
 - Mischievous enemy: moves orbs from one holder to another, possibly undoing the player work
- Echo holders:
 - Teleport holder: player gets teleported to certain position
 - Freeze creatures

- Combat system: player can fight enemies
- Red orbs (power ups)
 - Dash ability
 - Damage level increase
 - Speed increase
- Sound effects
- Music
- Artwork and sprites
- HUD

5.2.2 Editor

- Edit/play modes: edit mode for building levels and play mode for testing
- Save/load
- Keyboard shortcuts

5.3 Stretch Features

5.3.1 Main Gameplay

- Cutscenes

5.3.2 Editor

- Undo/redo
- Generate level skeleton
- Level validation system
- Editor customization

6. Look and Feel

6.1 Visual Style

The overall look and visual aesthetic of Echo Orb is minimalistic yet expressive. It has a 2D pixel art style which is both cartoony and expressive, with colour and animation bringing the world to life. The pixel art style was utilised in early 2D games due to hardware limitations but is still seen today in many modern titles and has become its own unique style. One such example that is an inspiration for Echo Orb is Celeste (see Figure 3).

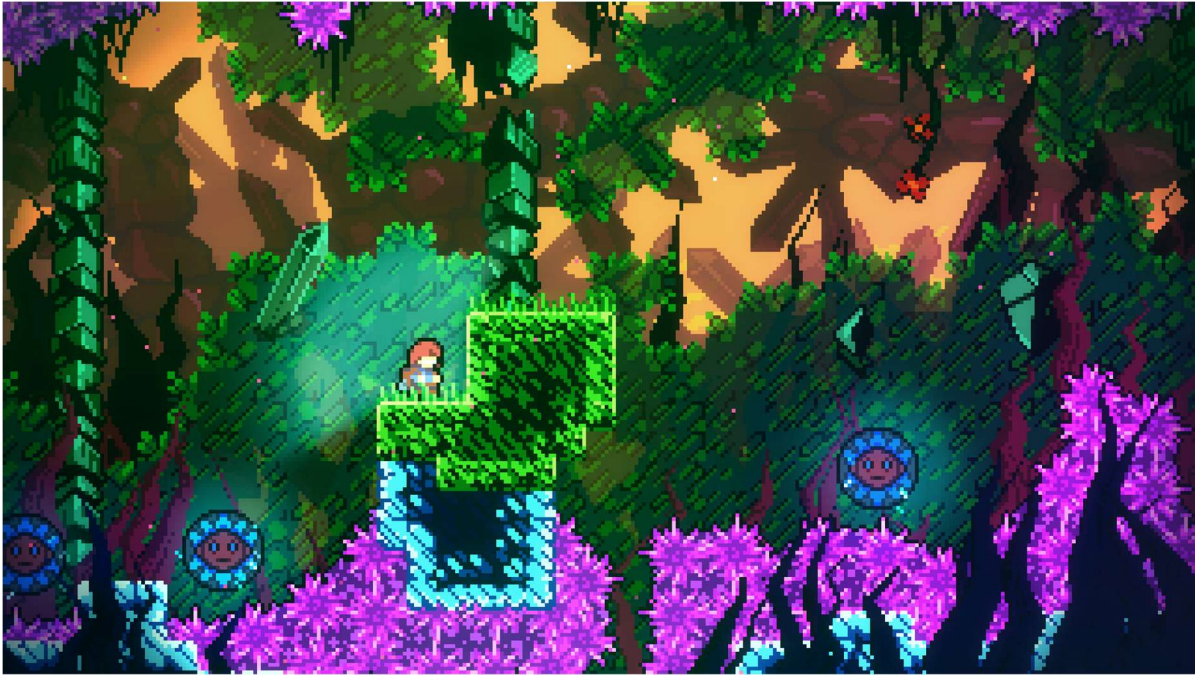


Figure 3. Screenshot from Celeste (Maddy Makes Games Inc., Extremely OK Games, Ltd, 2018)

The main colour palette will use contrasting colours like teals and oranges for a visually appealing style, and to help key elements stand out. Effective use of lighting will be needed to illuminate darker areas. Lamps, torches, and the faint glow that emits from the orbs will provide sources of light guiding the player through the dim depths of the dungeons. Colour will also be used to convey emotion, inspired by Celeste. In Celeste, scenes with lively or hopeful moments use bright and colourful tones, while the darker, gloomier moments use a much more muted palette and darker colours. Echo Orb will set the mood and atmosphere in a similar way using shifts in colour and lighting.

Another art inspiration for Echo Orb is Diamond Rush (see Figure 4). The adventurous yet simple visual tone is something I would like to incorporate into the design of Echo Orb. I was also very inspired by Diamond Rush's temple theme and level design.

The UI's look and feel will be consistent with the look and feel of the gameplay itself, maintaining a similar colour palette and overall visual style. In game, the HUD must stand out without distracting from the main gameplay. Layouts will be carefully designed with usability in mind, while also maintaining a visually appealing aesthetic. This is particularly important for the level editor, which utilises a lot of UI elements. Geometry Dash's editor demonstrates a well-designed layout that is also visually appealing (see Figure 5).



Figure 4. Diamond Rush gameplay (The Spriters Resource, 2024)



Figure 5. Geometry Dash level editor (Geometry Dash Wiki, 2025)

6.2 Audio Style and Mood

The audio style will work with the visuals to create an overall sense of mystery and wonder. Moments of high intensity and danger will be accompanied by fast paced upbeat music, while lower intensity moments such as exploration and puzzle solving moments with little chance of danger will be accompanied by calmer ambient music. There will be lots of sound effects for all the different actions and interactions. This will make gameplay feel alive and realistic. Sounds that play frequently in succession such as the sound of cannons shooting will use subtle changes in pitch to avoid sounds feeling repetitive and tedious.

Most of the artwork and some of the sound effects will be designed by me, and appropriate fonts and music will be carefully chosen from a third party.

7. Narrative, theme, and aesthetics

7.1 Story

Eli and his exploration team are deep underground, when Eli trips and falls down a deep hole, separating himself from the rest of the team. He sees a mysterious blue glow in the distance and decides to follow it. It leads to an ominous looking door. Eli is scared yet curious, and the moment he walks through the door it slams shut behind him. Terrified Eli looks around to discover he is inside what appears to be some sort of dungeon. The door won't budge, and Eli notices a golden receptacle, with an empty hemispherical cradle. As Eli ventures deeper into the vast chambers of the dungeon he finds a glowing orb emitting a faint blue light. To Eli's surprise, it does not fit the receptacle. He then finds more orbs, and smaller, similar looking receptacles that the orbs fit into. This gives Eli seemingly magical abilities allowing him to control the dungeon itself. He also finds a civilization of unusual creatures. When he gets the courage to talk to them, they tell him the story of The Golden Echo Orb. Legend says that The Golden Echo Orb is a magical item that was shattered thousands of years ago, and its pieces are long lost. Eli is intrigued by the story and sets out on a quest to recover the pieces and rebuild the Orb. Some of the creatures say that it's impossible, but Eli is determined. Eli uses the powerful force of the Echo Orbs to not only navigate but also control the seemingly enchanted chambers of the dungeon, and the interconnected systems within. When Eli finally recovers the orb and places it in the grand receptacle, the door opens, and a stairway appears taking him to the surface.

7.2 Setting and Characters

Eli: The protagonist, a curious and determined explorer

The Lani species: Ancient subterranean intelligent species Eli befriends

Bebe: Eli's Lani friend who speaks of the ancient stories and legends

The dungeon is a mysterious and enchanted place deep beneath the Earth's surface. Its shifting environment, controlled by the Echo Orbs, is vast and is made up of sprawling chambers. The faint blue glow of the Echo Orbs creates a feeling of unease. Unusual creatures inhabit this world and roam the chambers. Some are hostile and others are friendly.

7.3 Tone and themes

The tone of the story is mysterious and tense, with a sense of unease as Eli is separated from the rest of the team. It slowly shifts towards wonder and hope, as Eli becomes more curious and courageous. The central themes are discovery, hope, and exploration.

7.4 Aesthetics

The MDA framework describes the Mechanics, Dynamics, and Aesthetics of a game. Aesthetics are the emotional responses evoked in the player. Echo Orb attempts to give the player a sense of wonder and intrigue as they uncover hidden secrets and untold truths about the environment, setting, ancient legends, and mysterious creatures of the Echo Orb world.

8. Technical Approach

8.1 Game Engine:

The game will be made Unity 6. I chose Unity because it is the game engine I am most familiar with, and I think it is well suited for this project. I am using Unity 6 as it is the most recent and up to date version.

8.2 Programming Language:

The game will be programmed in C# as this is what Unity uses.

8.3 Key Systems:

8.3.1 Core Architecture

A modular component-based architecture will be implemented to allow for reusability and scalability. Echo Holders will follow a polymorphic design, allowing for new holder types to be easily added. An event-driven system will be used for the Echo system. When holders are activated, a global signal is broadcasted and all holders subscribed to this event perform their assigned behaviour. This approach keeps in line with the modular and expandable design. Finite state machines will be used for enemy/NPC logic, and player/character animations, defining transitions between states.

8.3.2 Level Editor Implementation

The Level Editor will make use of Unity's Tilemap System to construct and render game levels. A toggleable grid-based layout will be used for object placement. Player built levels will be exported to JSON using a custom serializer to record level data such as tile data, objects, Echo Holder states, NPC spawn points, etc. Imported levels will have to be validated to ensure critical components such as player spawn point are included to prevent invalid levels.

8.3.3 Puzzle Logic and Echo System

Puzzle logic will be tested extensively. Since players can create their own puzzles, the risk of unintended logic gaps is increased, so the underlying systems must be robust.

8.3.4 Game AI

Game AI techniques will be used to create intelligent and dynamic behaviours for characters and enemies. Finite state machines will define transitions between states such as idle, follow, attack, etc. Algorithms such as A* will be used for pathfinding.

8.3.5 Performance and Optimisation

Careful consideration will be given to optimisation and overall performance. The Unity profiler will be used to track performance and identify potential bottlenecks. An object pooling system will be implemented to manage frequently spawned objects such as projectiles and Echo Orbs dropped by enemies, reducing runtime instantiation costs.

8.3.6 UI and Accessibility

Careful considerations will be made when designing the UI, making sure to adhere to good UI and UX design principles. Controls will be re-bindable, and colour palettes will be selected carefully for with accessibility in mind.

8.4 Tools:

- Visual Studio 2022 for C# programming
- GIMP for creating artwork, sprites, and animations
- Audacity for creating and editing sound effects

8.5 Data Storage:

- Unity PlayerPrefs are used for storing basic game data and preferences, such as levels completed, current progress, user set options, etc.
- Custom level data is stored in JSON format.

8.6 Version Control:

A GitHub repository with frequent commits is being used for version control.

8.7 Deployment Platform:

The game will be deployed on itch.io, for Windows PCs.

9. Literature and Research Summary

9.1 Emergent Gameplay

Emergence is the process by which something becomes apparent or comes into existence, like an animal emerging from behind the trees, or an idea emerging from inspiration. In the context of game design emergence is the combination of rules or mechanics, creating new situations. Simple mechanics can be combined to create complex or interesting systems. Such systems may be unintended or unexpected by the developer.

The idea of designing with emergent gameplay in mind may seem somewhat paradoxical, as it essentially suggests that the developer is designing something unintentional. As stated by Rashidi and Bagher (2023), emergence can happen only after a system is set in motion. However, the specific emergent situations the player may discover are not specifically designed for, rather the developer has created a rich set of modular, flexible, and interactive systems that are designed to interact with each other in a way that leads to countless possibilities and adds significant depth to gameplay. Rashidi and Bagher (2023) also suggest that this illuminates the importance of playtesting and prototyping.

Emergent behaviour can be seen in cellular automata, where a set of simple rules can lead to complex behaviour. A cellular automaton is a set of rules defining the state of cells generally on a grid, based on the state of the cell itself and its neighbouring cells. A well-known example is John Conway's *Game of Life* (Martin, n.d.). In the *Game of Life*, the cells have two states: alive or dead, represented visually as black or white squares. The set of rules are applied repeatedly:

- Dead cells with three neighbours become alive
- Alive cells with zero or one neighbour die
- Alive cells with two or three neighbours survive
- Alive cells with four or more neighbours die

While these rules by themselves seem very simple, when combined they display complex and unpredictable behavioural patterns.

There are two basic game structures that make up a video game - emergence and progression. The idea was first proposed by Juul (2005) and has been expanded on by other like Tekinbas and Zimmerman (2004). Some games lean more into the emergent side, and others into the progression side, but most have a combination of the two. Progression games involve completing a series of predefined actions in order to progress. Gameplay of this type is much more controlled by the designer. Progression structures are not a bad thing, but purely progressive gameplay may make the player feel like they are simply playing an interactive story with little freedom to do whatever they want. Emergent games usually have a much greater replay value than progressive. Juul also suggests that traditional games like board and card games fall into the emergent category. The progressive model was only introduced in the creation of video games.

Emergent Gameplay in Echo Orb

In *Echo Orb* I intend to implement emergent gameplay by introducing several simple puzzle mechanics that can be combined to create more interesting puzzle and gameplay situations. The game's emphasis on creating your own levels will allow players to test out many different combinations. The orb and holder mechanic specifically is one that is simple by itself. Each echo holder type has its own function. It does just one action, like toggling invisible walls, or firing a cannon. There is not much challenge in a puzzle with just one echo holder, but making a puzzle utilizing several echo holders of different types can lead to emergence; things that I as the developer may not have even thought of. The modular nature of these

simple mechanics will allow the player to be creative in both their solutions to the puzzles, and in creating their own puzzles.

9.2 Human-Computer Interaction

Human-Computer Interaction (HCI) is the study of how humans interact with computers. It is inherently linked with User Interface (UI) and User Experience (UX) which deal with how we can make this interaction as intuitive and frictionless as possible for the user. HCI, UI, and UX combined encompass ease of use, efficiency, and accessibility of digital systems. HCI considerations are essential in the design of user interfaces, and various frameworks have been proposed which have formed the principals and approach to effective UI design.

One of the main contributions in the area of HCI is Nielsen's Usability Heuristics. Nielsen (1994) proposed a list of general design principles which he suggested are to be used as general guidelines rather than strict rules. Nielsen's heuristics emphasise error prevention, consistency, and allowing the user a sense of control. Some ways of utilising these guidelines include supporting undo/redo actions, allowing the user to cancel an action, and maintaining consistency both across the overall user interface, and with similar products, as this is what the user is already used to. For example, when choosing icons to be used in the user interface, it is usually best practice to choose icons that most users are already familiar with. Common actions such as the save, undo, and redo most often use similar icons across different applications. Using a completely different icons than those of similar products could be confusing to the user, as they would have to learn a new symbol for a familiar action. Shneiderman and Plaisant (2010) compliment Nielsen's work with Shneiderman's Eight Golden Rules. There is some overlap between the two, but Shneiderman and Plaisant add that we should reduce the user's short-term memory load. This involves keeping our user interfaces simple and not requiring the user to remember complex sequences of actions, helping us to maintain an intuitive interface, and ease of use.

Both models emphasise the importance of allowing shortcuts for the experienced user. The user interface should be simple to use with just a few basic rules but should also include shortcuts to allow the user to perform frequently used operations much more efficiently. One kind of shortcut is keyboard shortcuts which perform an action with a single keystroke (or key combination) that may have otherwise taken multiple steps/clicks. These are especially needed for the most common commands that are used repeatedly. Linked with shortcuts is customisability. Allowing the user to tailor the user interface to their needs allows for greater ease of use and personalisation. The user can prioritize the tools they need in a way that suits them best. Combining this with shortcuts allows for custom shortcuts that suit each user individually. In video games specifically, allowing the player to customize the controls allows for a much better user experience. Restricting the player to the default controls may lead to the user feeling frustrated and may lead to accessibility issues for some players. Other accessibility considerations include choosing high contrast colour schemes (and perhaps including a colour-blind mode, which may make use of textures or patterns where colours are used to distinguish certain elements), including alternative text for images, and including captions for video or audio media.

HCI in Echo Orb

Due to the nature of Echo Orb's Level editing feature, the game will be rich in user interfaces, and as such, careful consideration will be made with regards to the user experience. The style will remain consistent across the game; icons will be familiar, resembling those used in similar products; the level editor interface will be kept simple as possible and laid out in a way that makes sense intuitively to the user, without overloading them with new things to learn; the editor will support undo and redo actions; shortcuts will be included to allow experienced users better efficiency and comfort; and the user will be given a sense of freedom by allowing them to adjust the game and editor controls to their liking.

9.3 User Generated Content

User generated content (UGC) is a broad term that encapsulates all content created by the user, rather than the developer. This may be text, images, videos, or anything else created by the user. Common examples of UGC seen in video games include custom assets, mods, and levels. UGC allows the player to actively contribute to the games they play, to be creative, and may even allow the player to find new and interesting ways to play, possibly leading to emergence.

UGC opportunities in games has benefits for the player. Duan et al. (2022) suggest that giving players UGB opportunities enhances playability and longevity. They distinguish between different types of content, as well as content editors, including level editors, gameplay editors, game editing engines, and community-made game editors, all of which allow player creativity, and allow the player to find ways to make gameplay feel new and varied.

Video gameplay modification is a very popular example of UGC. Such modification may or may not be an intentional part of the gameplay experience. Paul (2025) defines gameplay modification as “the alteration of the experience of playing the game from the base or ‘intended’ gameplay experience”. The lines are blurry on what does or does not count as game modification. Using a wiki or forum with instructions on how to complete a part of a game may alter the intended experience, but does this count as game modification? The same can be said about changing the default controls, or utilising exploits and glitches. For the purposes of this review, I will focus on software modification ie., altering a game’s code. These are often referred to as “Mods”. Mods can bring many different types of alterations to a game, including but not limited it altered graphics, bug fixes, and adding entirely new gameplay elements. Modding became popular with the release of *DOOM* (1993). The game was intentionally built with modding in mind and allowed much greater modding capabilities than had been seen in previous games due to the restructuring of game data and released documentation of file formats. This new restructured design of the game data led to what we know today as the “game engine” (Sotamaa, 2022). Several hugely popular games started out as mods. For instance, the game *Counter-Strike* (Valve Corporation, 2000) was originally a mod for *Half-Life* (Valve Corporation, 1998).

Duan et al. (2022) discuss four categories of video game UGC editors: level editors, gameplay editors, game editing engines, and community-made game editors. Level editors allow players to use existing game elements to create new levels within the confines of the base game rules, expanding the game experience rather than creating a new one. Gameplay editors however do allow for the player to make their own rules, allowing for creation of rules and modification of the existing ones. Gameplay editors generally give the player more freedom than level editors, allowing for a broader range of UGC possibilities, for example new game modes. Game editing engines are like gameplay editors, but they allow for external assets to be brought into the game. With this type of editor, games can be made look totally different from the base game. There are copyright concerns associated with this kind of editor, however. As a result, the UGC associated with this type of editor is usually not distributed within games themselves and are instead distributed by the creators instead in the form of mods. Community-made game editors are not found within games at all. They are instead built and maintained by the community. All forms of UGC editors enhance the longevity of their respective game.

UGC in Echo Orb

Players will be encouraged to create UGC in Echo Orb using the in-game editor, where creativity and experimentation will be encouraged. The editor will allow brand new levels to be built using the base game elements. Using the categories defined by Duan et al., it would be classified as a level editor. This will add a lot to the replay value, allowing for players to expand the game and find their own new ways to play.

10. Scope and Feasibility

Feature/System	Category	Included in vertical slice?	Rationale/Notes
Game			
Player movement	Core	Yes	Demonstrates core gameplay
Pickup orb	Core	Yes	Demonstrates core gameplay
Place orb in holder	Core	Yes	Demonstrates core gameplay
Ghost walls	Core	Yes	Demonstrates core gameplay
Cannons	Core	Optional	Demonstrates core gameplay, but may be left until a later stage if time is limited
Teleport	Secondary	No	Not required at this stage
Combat	Secondary	Optional	Basic combat system to be included in vertical slice.
Echo Orb/holder system	Core	Yes	Demonstrates core gameplay
Red orbs (power-ups)	Secondary	No	Not required at this stage
Stairs transition between levels	Core	Yes	Demonstrates core gameplay
Enemy AI	Core	Optional	Basic AI for vertical slice
~ 5 premade levels	Core	Optional	1-2 required for vertical slice
Sound effects	Secondary	Optional	Improve player experience but not required for functionality
Music	Secondary	Optional	Improve player experience but not required for functionality
Cutscenes	Stretch	No	Can be implemented later
Artwork & sprites	Secondary	Optional	Basic sprites required but will be improved upon later
HUD	Secondary	Optional	Basic HUD required but may be improved upon later
Menu	Core	Yes	Basic menu required but may be improved upon later
Restart level	Core	Yes	Needed to prevent soft lock
Editor			
Place tiles	Core	No	Will be implemented in Semester 2
Place objects (orbs, echo holders, enemies, walls)	Core	No	Will be implemented in Semester 2
Remove objects	Core	No	Will be implemented in Semester 2
Edit/Play mode	Secondary	No	Will be implemented in Semester 2
Undo/Redo	Stretch	No	Will be implemented in Semester 2
Generate level skeleton	Stretch	No	Will be implemented in Semester 2
Level validation system	Stretch	No	Will be implemented in Semester 2
Editor UI	Core	No	Will be implemented in Semester 2

11. Production Plan

11.1 Methodology

I am using the scrum methodology to manage the project. The work is split into two-week sprints, with a review of my progress and a meeting with my advisor at the end of each sprint. This breaks the workload into smaller more manageable sized parts. I have a backlog with all required tasks and a sprint backlog with all current tasks. Each task is assigned a deadline which may be adjusted if more or less time is required.

11.2 Tools

I am using Trello to manage my Kanban style scrum board. I have four columns: product backlog, sprint backlog, in progress, and done. At the beginning of each new sprint, I evaluate my product backlog and move a reasonable number of tasks to the sprint backlog section. Over the course of the sprint, I move items from the sprint backlog to the in-progress section as I start new tasks. When I complete a task, it gets moved to the done section. Trello allows me to add deadlines to the tasks which helps me keep on track. It also allows me to add additional notes and checklists, where I sometimes add subtasks to break down large tasks into smaller more manageable ones. Trello's tags also allow for another level of organization.

I am using Miro to create flowcharts and diagrams. (See figure 2 - an example Miro diagram). Miro allows me to brainstorm and visualise my ideas. I am using YouTube to post dev log videos at the end of each sprint. This allows me to keep a record of my progress and my thoughts about the project, and to share my overall progress in a concise video. Slack is being used for communication between myself and my advisor, and Zoom is being used for meetings.

11.3 Sprint Schedule

Goal	Date
▪ Submit project proposal	18 th September
▪ Assemble development tech ▪ Setup Word doc ▪ Complete report section 10 ▪ Setup Trello ▪ Begin looking into research topics ▪ Create dev log	9 th October
▪ Report sections 1, 9, 10 ▪ Create Miro board ▪ Update Trello and create dev log	23 rd October
▪ Complete and upload ethics checklist	25 th October
▪ Clarify game concept ▪ Draft report sections 2, 3, 4, 5, 9, 11 ▪ Integrate Miro diagrams in document ▪ Create first playable prototype demonstrating a mechanic or system ▪ Update Trello and create dev log	6 th November
▪ Draft report sections 2, 3, 4, 5, 9, 11 ▪ Integrate Miro diagrams in document ▪ Create first playable prototype demonstrating a mechanic or system ▪ Setup GitHub repo ▪ Update Trello and create dev log	20 th November
▪ Complete report section 9	4 th December
▪ Complete document ▪ Work on key mechanics	18 th December
▪ Finalise vertical slice ▪ Submit semester one report	1 st January

Submit recorded demo	
- Prepare presentation slides - Present semester one presentation and demo	6 th January

11.4 Gantt Chart

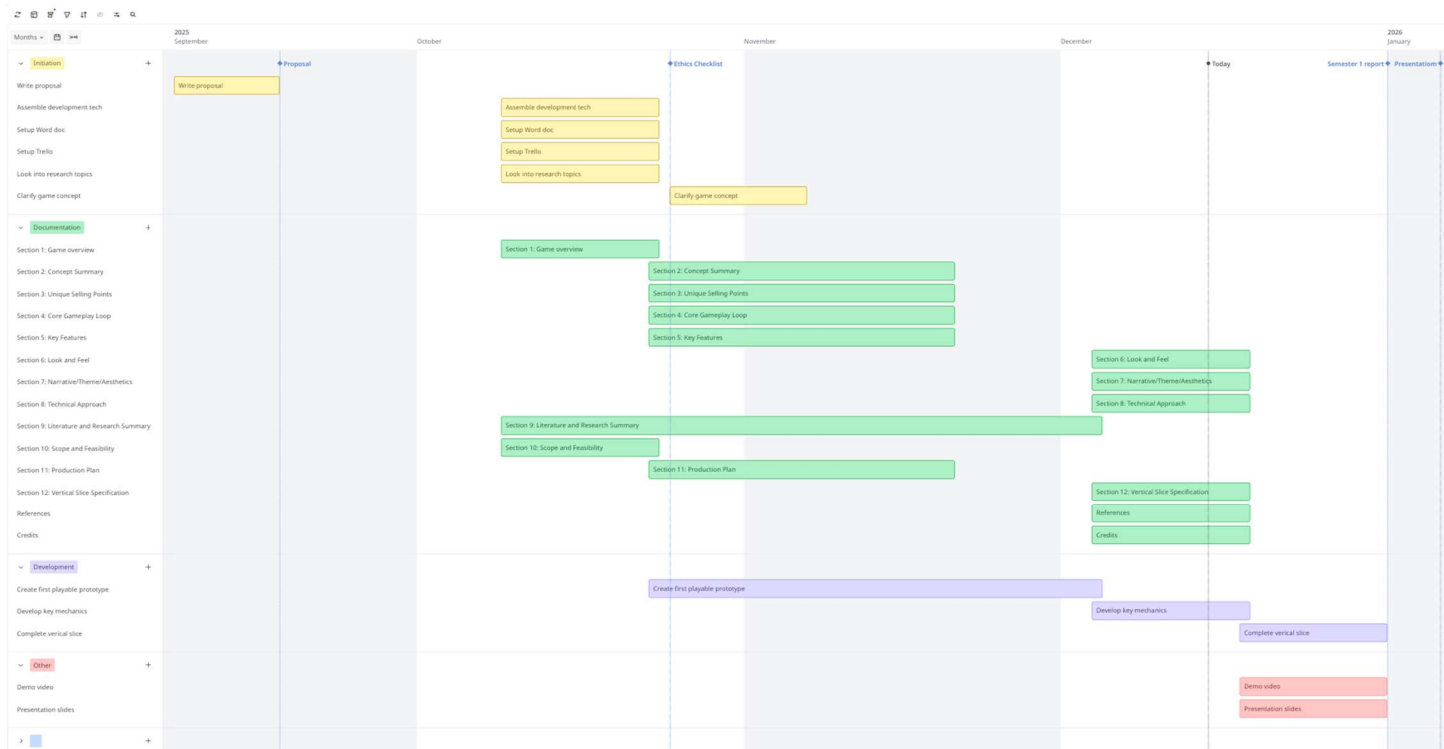


Figure 6. Gantt Chart created with Miro

11.5 Risk Analysis

- Time management: Due to the large scope of this project and the complex features to be implemented, time will have to be managed wisely to avoid falling behind on sprint deadlines. To mitigate this, I will limit sprint backlogs to an achievable level, regularly review and revise deadlines and priorities, and plan with catchup time in mind.
- Technical complexity: Features such as interconnected puzzle logic, NPC behaviours, and a working level editor may be quite challenging to implement. To mitigate this, I will build features in small increments and prioritize core mechanics before secondary or stretch goals.
- Trouble balancing difficulty: Levels will have to be challenging enough so that the player is not bored, yet simple enough so the player is not overwhelmed and not knowing how to progress. This will be managed with careful tuning based on playtesting results.

11.6 Testing Plan

I plan to have several people playtest my prototype over the course of its development. When a major feature gets added, I will get play testers to try it out, analyse their gameplay experience, ask for feedback, evaluate their responses, plan adjustments, and make adjustments accordingly. After adjustments have been made, ideally, I will get players to test again. Towards the vertical slice deadline, I will attempt to do some more in depth play testing.

I will ask friends, family members, classmates, and possibly other students to partake in playtesting my project. I will particularly seek for people in the target audience of the game and try to avoid bias where possible.

12. Vertical Slice Specification

The vertical slice will be a prototype demonstrating a few key mechanics and the core gameplay loop. There will be two playable levels demonstrating the echo orb/holder system, ghost walls, cannons, and one simple enemy.

12.1 Mechanics and systems

The player can move around the levels and collides with walls and other objects.

The Echo Orb activation system will be implemented using Unity Events, with Echo Holders listening for an activation event. For an action to occur, however, the holder must be powered. The player can pick up orbs and place them in or remove them from the holders. Once a holder is powered the player may activate it from anywhere in the current level. The holders follow polymorphic design, with different types of holders extending on the base holder class, allowing for new holder types to be easily added. The vertical slice will contain two holder types: Ghost Wall Echo Holder, and Cannon Echo Holder.

Ghost walls are a core gameplay mechanic, and as such will be implemented. Through activation of a Ghost Wall Holder, ghost walls may be toggled “on” or “off”. When on, the walls act as a solid barrier blocking entities (such as the player or enemies) from passing through. When toggled off they will become partially transparent and entities may walk through them. Different coloured ghost walls will be associated with different Ghost Wall Echo Holders. The vertical slice demonstrates two different coloured walls.

Cannons may be activated by a Cannon Echo Holder. When activated a projectile will be shot from the cannon. The projectiles damage both the player and enemies.

The enemy has very basic AI. If the player is near the enemy and is in the enemy’s line of sight, the enemy will simply move towards the player’s location. The enemy attacks the player when in range and takes damage from cannon projectiles. The enemy drops an orb on death.

12.2 Levels

Two basic levels will be included to demonstrate all the above mechanics and systems. One level will show the mechanics in isolation, and the other will combine them to demonstrate how the mechanics and systems work together to create puzzle scenarios. The levels will also demonstrate the game objective (reaching the level goal) and lose conditions (dying due to loss of health).

12.3 Temporary Assets

Most if not all the art assets in the vertical slice will be prototypes and will be replaced later on with more polished assets. Pixel art sprites will be made by me for wall tiles, floor tiles, player, enemies, orbs, and holders. Basic Unity shapes will be used for the remaining visuals.

Sound effects will be included for orb activation, powering/un-powering holders, and player damage. These sound effects will be sourced online.

12.4 Known Exclusions

The following will not be included in the vertical slice but will be worked on in Semester 2, along with the expansion of all mechanics, systems, levels, and assets mentioned above.

- Level editor
- Level validation
- Level serialisation
- Complex enemy AI

- Menu screens (pause, settings, etc.)
- More Echo Holder types
- More enemy/NPC types
- Red Orbs (powerups)
- Cutscenes and narrative elements

References

- Duan, H., Huang, Y., Zhao, Y., Huang, Z. and Cai, W. (2022). User-Generated Content and Editors in Video Games: Survey and Vision. *2022 IEEE Conference on Games (CoG)*. doi: <https://doi.org/10.1109/cog51982.2022.9893717>
- Geometry Dash Wiki (2025) *Build Toolbar*. Available at: https://geometry-dash.fandom.com/wiki/Level_Editor#Toolbars [accessed: 18 December 2025]
- id Software (1993) *DOOM* [Video game]. PC. id Software
- Juul, J. (2005). *Half-real: Video Games between Real Rules and Fictional Worlds*. [online] Cambridge (Massachusetts, USA): The Mit Press. Available at: <https://ebookcentral.proquest.com/lib/sae/reader.action?docID=6401867&ppg=8> [accessed: 04 November 2025]
- Maddy Makes Games Inc., Extremely OK Games, Ltd (2018) *Celeste* [Video Game]. PC. Maddy Makes Games Inc.
- Martin, E. (n.d.). *Play John Conway's Game of Life* [online]. Available from: <https://playgameoflife.com/> [accessed: 05 November 2025]
- Nielsen, J. (1994). *Usability Engineering*. Amsterdam: Morgan Kaufmann.
- Paul, L.E. (2025) *Tailoring Gameplay: Exploring Player Perspectives of Digital Game Modification*, Master's thesis, University of Victoria.
- Rashidi, H. and Bagher, L.P.M. (2023) 'On Complex Systems and Structure of Emergence in Games- A Survey', *Journal of Computer & Robotics*, 28(2), p.1. Available at: https://journals.iau.ir/article_700397_4ae4037303b673b67983d21e32884590.pdf [accessed: 05 November 2025]
- Shneiderman, B. and Plaisant, C. (2010) *Designing the User Interface: Strategies for Effective Human-Computer Interaction*. 5th ed. Pearson
- Sotamaa, O. (2022). *Modding*. In Grabarczyk, P. (ed.), *Encyclopedia of Ludic Terms* (Spring 2022 Edition). Available at: <https://eolt.org/articles/modding> [accessed: 02 December 2025]
- Tekinbas, K.S. and Zimmerman, E. (2004). *Rules of Play: Game Design Fundamentals*. MIT Press.
- The Spriters Resource (2024) *Angkor Wat. Level 1*. Available at: <https://www.spritters-resource.com/mobile/diamonrush/asset/215817/> [accessed: 18 December 2025]
- Valve Corporation (1998) *Half-Life* [Video Game]. PC. Valve Corporation.
- Valve Corporation (2000) *Counter-Strike* [Video Game]. PC. Valve Corporation.

Credits

Activation sound from Leszek_Szary on Pixabay: <https://pixabay.com/sound-effects/teleport-14639/>

Place Orb sound from Leszek_Szary on Pixabay: <https://pixabay.com/sound-effects/scale-c7-106132/>

Take Orb sound from Leszek_Szary on Pixabay: <https://pixabay.com/sound-effects/scale-g6-89174/>

Player damage sound from cabled_mess on Pixabay: <https://pixabay.com/sound-effects/hurt-c-08-102842/>

Pickup Orb sound from twohoursago on Pixabay: <https://pixabay.com/sound-effects/glass-bottle-clink-90671/>

Font from Catterio Sylt on dafont.com: <https://www.dafont.com/ari-w9500.font?text=Echo+Orb+A&back=bitmap>